Diseño y desarrollo de servicios web

Código: GA7-220501096-AA5-EV03

Javier Alejandro Toca Caro

Servicio nacional de aprendizaje (SENA)

Análisis de sistemas de software (2977395)

John Alejandro Niño Tambo

Sogamoso, Colombia

15 febrero 2025

Introducción

En la actualidad, los servicios web representan un componente fundamental en el desarrollo de aplicaciones modernas, ya que permiten la comunicación eficiente entre clientes y servidores mediante protocolos estandarizados. En este contexto, el proyecto Gastronomía Universal se enfoca en el diseño y desarrollo de una API REST que simula el funcionamiento de un restaurante digital.

El objetivo principal del proyecto es implementar servicios web que permitan gestionar información relacionada con platos, carrito de compras y pedidos, aplicando buenas prácticas de programación y una arquitectura organizada por capas. A través del uso de métodos HTTP como GET, POST y DELETE, se garantiza una interacción clara, estructurada y coherente entre el usuario y el sistema.

Asimismo, el desarrollo del proyecto permite fortalecer competencias en diseño de endpoints, validación de datos, manejo de códigos de estado HTTP y separación de responsabilidades dentro del código. De esta manera, Gastronomía Universal no solo cumple con los requerimientos funcionales planteados, sino que también demuestra la correcta aplicación de principios fundamentales en el desarrollo de servicios web.

Objetivo

Diseñar e implementar servicios web bajo el modelo de arquitectura REST para el proyecto Gastronomía Universal, permitiendo la gestión eficiente de platos, carrito de compras y pedidos mediante el uso adecuado de métodos HTTP, validaciones de datos y una estructura organizada por capas.

Diseño de los servicios

En esta parte se explica que los servicios web no se crean al azar, sino que deben responder a necesidades reales del sistema.

Por ejemplo, en un sistema de restaurante digital como Gastronomía Universal, el sistema necesita:

- Registrar pedidos que realizan los clientes.
- Calcular automáticamente el total de la compra.
- Permitir consultar los pedidos realizados por un usuario.

Esto significa que cada servicio que se desarrolla debe tener un propósito claro dentro del funcionamiento del sistema.

POST /api/pedidos

Este endpoint utiliza el método POST porque su función es crear un nuevo recurso, en este caso, un pedido.

Este servicio realiza varias acciones importantes:

1. Recibe el usuario
   Se valida que el usuario exista en el sistema antes de crear el pedido.
2. Recibe la lista de productos
   El cliente envía los productos que desea comprar junto con sus cantidades.
3. Calcula automáticamente el total
   El sistema toma el precio de cada producto, lo multiplica por la cantidad y suma todo para obtener el total.
   Esto evita que el cliente manipule el valor final.
4. Guarda el pedido
   Finalmente, el pedido se almacena en la base de datos con su información completa.

Nombre de enpoint

Cuando hablamos del nombre del endpoint, nos referimos a la ruta que identifica un servicio dentro de la API. Es la dirección específica a la que el cliente envía una solicitud para ejecutar una acción.

En este caso, el nombre del endpoint es:

/api/pedidos

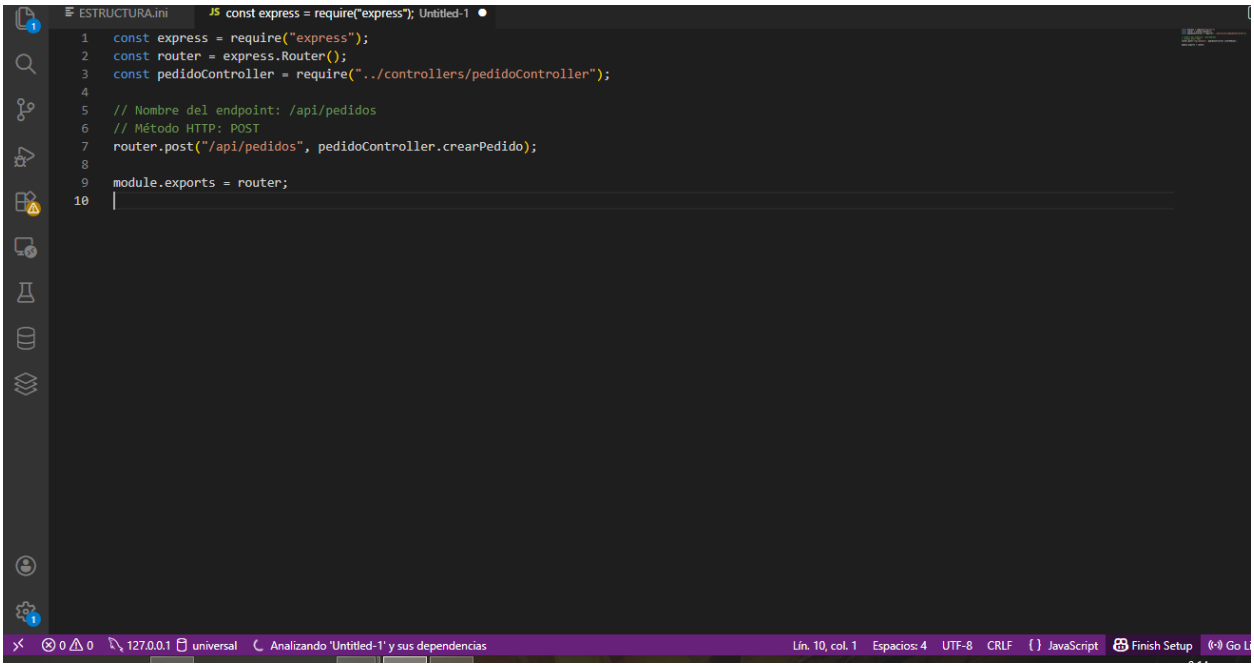
Esta ruta indica que estamos trabajando con el recurso “pedidos” dentro de nuestra API.

Ahora bien, un endpoint siempre debe ir acompañado de un método HTTP, que define qué acción se va a realizar sobre ese recurso.

En este ejemplo:

- Nombre del endpoint: /api/pedidos
- Método HTTP: POST
- Descripción: Crea un nuevo pedido en el sistema

El método POST se utiliza porque estamos creando un nuevo recurso, es decir, un nuevo pedido. Si fuera para consultar información, usaríamos GET; si fuera para eliminar, usaríamos DELETE.



```javascript
const pedidoService = require("../services/pedidoService");

const crearPedido = (req, res) => {
  const { idUsuario, productos } = req.body;

  const resultado = pedidoService.crearPedido(idUsuario, productos);

  if (resultado.error) {
    return res.status(resultado.status).json({
      mensaje: resultado.error
    });
  }

  return res.status(201).json({
    mensaje: "Pedido creado correctamente",
    pedido: resultado.pedido
  });
};

module.exports = {
  crearPedido
};
```

```javascript
const express = require("express");
const app = express();
const pedidoRoutes = require("./routes/pedidoRoutes");

app.use(express.json());
app.use(pedidoRoutes);

app.listen(3000, () => {
  console.log("Servidor ejecutándose en http://localhost:3000");
});
```

Aquí podemos ver que el endpoint se define en el archivo de rutas usando router.post('/api/pedidos'). Esto significa que cuando el cliente envía una solicitud POST a esa ruta, se ejecuta el controlador que crea el pedido.

- Método HTPP

Un método HTTP indica la acción que el cliente quiere realizar sobre un recurso dentro de la API.

En otras palabras, el método HTTP define qué queremos hacer: consultar, crear, actualizar o eliminar información.

En el sistema Gastronomía Universal, los métodos HTTP permiten gestionar pedidos, usuarios y productos del restaurante digital.

- Métodos HTTP aplicados a Gastronomía Universal

POST – Crear información

Se usa cuando queremos registrar algo nuevo en el sistema.

- **POST /api/pedidos**

Se utiliza para crear un nuevo pedido el cliente envía el usuario y los productos, y el sistema calcula el total y guarda el pedido.

GET – Consultar información

Se usa para obtener datos del sistema.

- **GET /api/pedidos**

Permite consultar todos los pedidos registrados.

- **GET /api/pedidos/1**

Permite consultar un pedido específico por su ID.

PUT – Actualizar información

Se usa para modificar información existente.

- **PUT /api/pedidos/1**

Permite actualizar un pedido, por ejemplo, cambiar productos o estado.

DELETE – Eliminar información

Se usa para borrar datos del sistema.

DELETE /api/pedidos/1

Elimina un pedido del sistema.

```javascript
const express = require("express");
const router = express.Router();
const pedidoController = require("../controllers/pedidoController");

// GET - Consultar todos los pedidos
router.get("/api/pedidos", pedidoController.obtenerPedidos);

// GET - Consultar pedido por ID
router.get("/api/pedidos/:id", pedidoController.obtenerPedidoPorId);

// POST - Crear nuevo pedido
router.post("/api/pedidos", pedidoController.crearPedido);

// PUT - Actualizar pedido
router.put("/api/pedidos/:id", pedidoController.actualizarPedido);

// DELETE - Eliminar pedido
router.delete("/api/pedidos/:id", pedidoController.eliminarPedido);

module.exports = router;
```

```javascript
let pedidos = []; // Base de datos simulada

// GET todos
const obtenerPedidos = (req, res) => {
  res.json(pedidos);
};

// GET por ID
const obtenerPedidoPorId = (req, res) => {
  const pedido = pedidos.find(p => p.id == req.params.id);

  if (!pedido) {
    return res.status(404).json({ mensaje: "Pedido no encontrado" });
  }

  res.json(pedido);
};

// POST crear
const crearPedido = (req, res) => {
  const { idUsuario, productos } = req.body;

  if (!idUsuario || !productos) {
    return res.status(400).json({ mensaje: "Datos incompletos" });
  }

  const nuevoPedido = {
    id: pedidos.length + 1,
    idUsuario,
    productos,
    estado: "Pendiente"
  };
```

```javascript
const crearPedido = (req, res) => {
  const nuevoPedido = {
    id: pedidos.length + 1,
    idUsuario,
    productos,
    estado: "Pendiente"
  };

  pedidos.push(nuevoPedido);

  res.status(201).json({
    mensaje: "Pedido creado correctamente",
    pedido: nuevoPedido
  });
};

// PUT actualizar
const actualizarPedido = (req, res) => {
  const pedido = pedidos.find(p => p.id == req.params.id);

  if (!pedido) {
    return res.status(404).json({ mensaje: "Pedido no encontrado" });
  }

  pedido.estado = req.body.estado || pedido.estado;

  res.json({
    mensaje: "Pedido actualizado correctamente",
    pedido
  });
};

// DELETE eliminar
const eliminarPedido = (req, res) => {
```

```javascript
const actualizarPedido = (req, res) => {
  if (!pedido) {
  }

  pedido.estado = req.body.estado || pedido.estado;

  res.json({
    mensaje: "Pedido actualizado correctamente",
    pedido
  });
};

// DELETE eliminar
const eliminarPedido = (req, res) => {
  const index = pedidos.findIndex(p => p.id == req.params.id);

  if (index === -1) {
    return res.status(404).json({ mensaje: "Pedido no encontrado" });
  }

  pedidos.splice(index, 1);

  res.json({ mensaje: "Pedido eliminado correctamente" });
};

module.exports = {
  obtenerPedidos,
  obtenerPedidoPorId,
  crearPedido,
  actualizarPedido,
  eliminarPedido
};
```

```javascript
const express = require("express");
const app = express();
const pedidoRoutes = require("./routes/pedidoRoutes");

app.use(express.json());
app.use(pedidoRoutes);

app.listen(3000, () => {
  console.log("Servidor Gastronomía Universal en puerto 3000");
});
```

En Gastronomía Universal:

- GET → Consulta pedidos
- POST → Crea pedidos
- PUT → Actualiza pedidos
- DELETE → Elimina pedidos

En Gastronomía Universal utilizamos métodos HTTP para definir qué acción se va a realizar. Por ejemplo, usamos POST para crear pedidos, GET para consultar información, PUT para actualizar datos y DELETE para eliminar registros. Esto garantiza que nuestra API siga una arquitectura REST y esté bien estructurada.

Parámetros de entrada

Los parámetros de entrada son los datos que el cliente envía al servidor para que el servicio pueda ejecutarse correctamente.

En una API REST pueden enviarse:

- En el body (cuando usamos POST o PUT)
- En la URL (cuando usamos GET con :id)
- En los query params

Para metros para crear un pedido

- **POST /api/pedidos**

Se envían en el BODY en formato JSON

```
{
  "idUsuario": 1,
  "productos": [
    { "idProducto": 3, "cantidad": 2 },
    { "idProducto": 5, "cantidad": 1 }
  ]
}
```

idUsuario

- Tipo: Número
- Función: Identifica qué usuario está realizando el pedido.
- Es obligatorio.

producos

- Tipo: Arreglo (array)
- Función: Lista de productos que el cliente desea comprar.
- Es obligatorio.

Dentro de productos tenemos:

idProducto

- Tipo: Número
- Función: Identifica el producto del menú.

cantidad

- Tipo: Número
- Función: Cantidad del producto que se desea comprar.

```javascript
const crearPedido = (req, res) => {
  const { idUsuario, productos } = req.body;

  if (!idUsuario) {
    return res.status(400).json({
      mensaje: "El idUsuario es obligatorio"
    });
  }

  if (!productos || productos.length === 0) {
    return res.status(400).json({
      mensaje: "Debe enviar al menos un producto"
    });
  }

  res.status(201).json({
    mensaje: "Parámetros recibidos correctamente"
  });
};
```

Los parámetros de entrada son los datos que el cliente envía al servidor. En el caso de Gastronomía Universal, para crear un pedido necesitamos el id del usuario y la lista de productos con su cantidad. Estos datos se envían en formato JSON dentro del body de la solicitud POST.

Respuesta esperada

- Endpoint:
- **POST /api/pedidos**

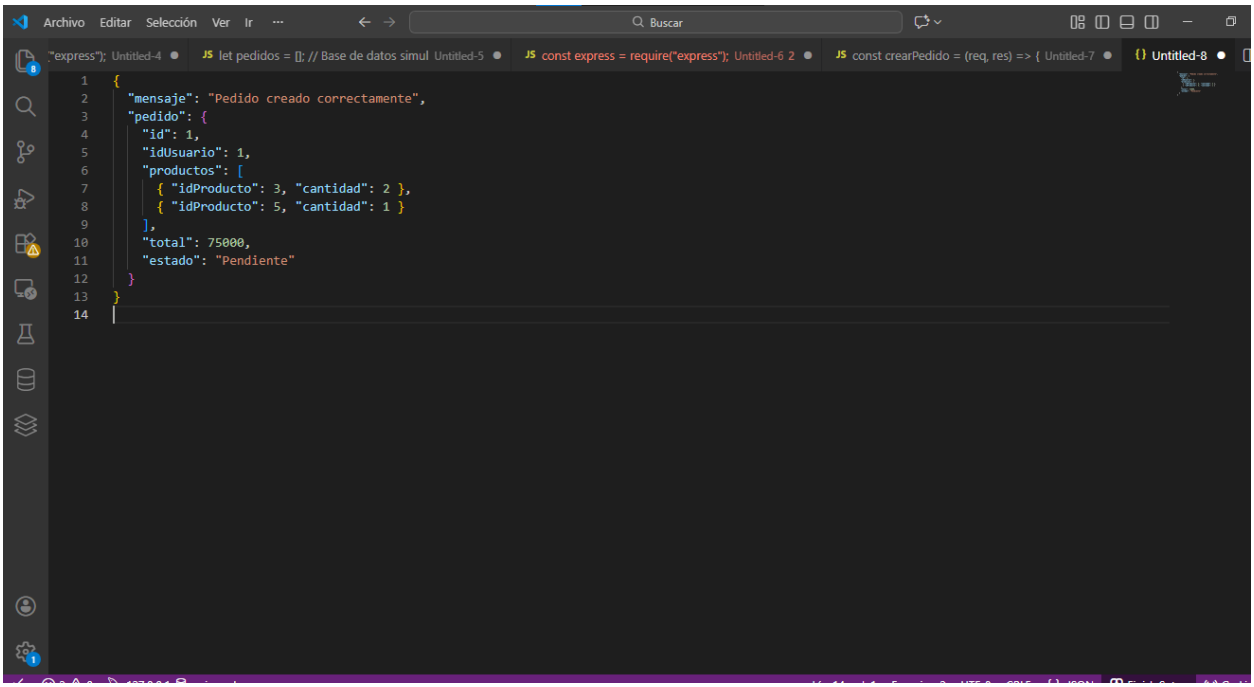
La respuesta esperada es la información que el servidor devuelve al cliente después de procesar la solicitud.

Indica:

- Si la operación fue exitosa
- Si hubo un error
- Qué datos se generaron

Código de estado:

- **201 Created**



- El pedido se creó correctamente.
- Se generó un ID único.
- Se calculó el total automáticamente.
- Se guardó el estado inicial del pedido.

Respuesta si faltan datos

- Código:

- **400 Bad Request**

{

"mensaje": "Datos incompletos"

}

Significa que el cliente no envió la información necesaria.

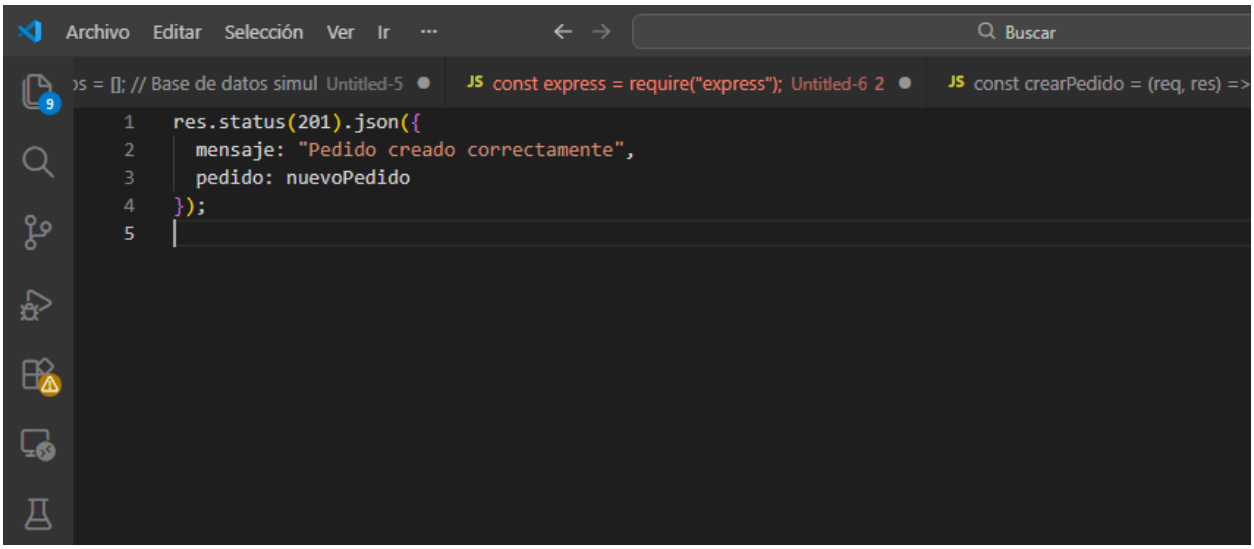
Respuesta si el usuario no existe

Código:

- **404 Not Found**

  Respuesta:

{
 "mensaje": "Usuario no encontrado"
}



La respuesta esperada del servicio POST /api/pedidos es un código 201 cuando el pedido se crea correctamente. El sistema devuelve un mensaje de confirmación y la información completa del pedido, incluyendo el total calculado automáticamente. Si ocurre un error, el sistema responde con códigos como 400 o 404 según el caso.”

Códigos de Estado Posibles

Endpoint:

POST /api/pedidos

Los códigos de estado HTTP indican el resultado de la operación que realiza el servidor.

201 – Created

Significa que el pedido fue creado correctamente.

Se devuelve cuando:

- El usuario existe
- Los productos existen
- Los datos son válidos
- El pedido se guardó correctamente

```
res.status(201).json({
  mensaje: "Pedido creado correctamente",
  pedido: nuevoPedido
});
```

400 – Bad Request

Significa que el cliente envió datos incorrectos o incompletos.

Se devuelve cuando:

- Falta el idUsuario
- No se envían productos
- La lista está vacía

```
if (!idUsuario || !productos) {
  return res.status(400).json({
    mensaje: "Datos incompletos"
  });
}
```

404 – Not Found

Significa que el recurso no existe.

Se devuelve cuando:

- El usuario no está registrado
- Algún producto no existe

```
if (!usuario) {
  return res.status(404).json({
    mensaje: "Usuario no encontrado"
  });
}
```

500 – Internal Server Error

Significa que ocurrió un error interno en el servidor.

Se devuelve cuando:

- Hay un fallo inesperado
- Ocurre un error en la lógica

```
try {
   // lógica del pedido
} catch (error) {
   res.status(500).json({
     mensaje: "Error interno del servidor"
   });
}
```

Código	Significado	Cuándo se usa
201	Created	Pedido creado correctamente
400	Bad Request	Datos incorrectos
404	Not Found	Usuario o producto no existe
500	Internal Server Error	Error inesperado

Los códigos de estado HTTP indican el resultado del servicio. En Gastronomía Universal usamos 201 cuando el pedido se crea correctamente, 400 cuando faltan datos, 404 cuando el recurso no existe y 500 cuando ocurre un error interno del servidor.

Link repositorio: https://github.com/javiertoca19-glitch/Repositorio-Proyecto/upload/main

Conclusión

En el desarrollo del sistema Gastronomía Universal, se aplicaron correctamente los principios del diseño y desarrollo de servicios web bajo arquitectura REST. Se definieron endpoints claros como POST /api/pedidos, utilizando métodos HTTP adecuados para cada operación, como GET, POST, PUT y DELETE.

Además, se implementó la validación de parámetros de entrada, el manejo correcto de respuestas esperadas y el uso adecuado de códigos de estado HTTP como 201, 400, 404 y 500. Esto garantiza que la API sea organizada, funcional y profesional.

La estructura del proyecto por capas (routes, controllers, services y models) permitió una correcta separación de responsabilidades, facilitando el mantenimiento y la escalabilidad del sistema.

En conclusión, el proyecto demuestra una implementación adecuada de servicios web, aplicando buenas prácticas, documentación clara y estándares REST, cumpliendo con los requisitos establecidos para el desarrollo de una API funcional y bien estructurada.